演示即上下文

DEMONSTRATION AS CONTEXT

给机器人一条人做事的录像，它能不能直接上手——这条线走到了哪一步、 三个和我们直觉相反的发现，以及我们准备补上的测量。

1. **听懂任务**
   语言
2. **找一条最像的演示**
   几何相似度
3. **按物体位置对齐**
   位姿变换
4. **照着重放**
   零训练

目录

CONTENTS

四个部分，从 " 这条路在问什么 " 讲到 " 我们准备补什么测量 " 。

01 这个方向在问什么

一条演示进来，机器人靠什么直接上手：找、对、放三步。

02 前人走到哪一步

三代做法的分界线，以及 " 上下文 " 这个词被用来指的三件不同的事。

03 三个没料到的发现

更多上下文反而更差、默认评测分辨不出五个点、三种错误会互相伪装。

04 接下来要做什么

三件能被别人复算的测量，以及需要团队配合的地方。

第一部分

01

先把问题说清楚，再谈文献。

这个方向在问什么

一句话：能不能让机器人"看一条人做事的录像"就上手，而且不为此重新训练一个模型。

- 要解决的不是"更准"，而是"新任务不用训"——这是它和主流端到端训练最根本的差别。
- 这条路的价值还在于"失败能定位"：找错了、对不齐、还是执行不了，可以分开查。
- 代价也很清楚：单臂、开环、演示和部署时物体的摆放不能差太多。

一条演示进来，机器人靠什么直接上手

整条链路只有四步，而且每一步都能单独出问题

1. **听懂要做的事**
   一句话 → 拆成抓、放、擦、拧这类动作名

   → 得到动作名

2. **在库里找一条最像的演示**
   当前物体点云 vs 库里每条演示首帧，取最近的一条

   → 得到一条演示

3. **把演示轨迹对到当前物体上**
   估物体位姿，整体搬运轨迹，再规划一条不撞的路

   → 得到一条路径

4. **照着重放**
   按演示的末端轨迹和夹爪开合重放；这一段开环

   → 直接执行

为什么值得研究

新任务不用训

加一条演示就是加一行数据，不更新任何参数。

失败能定位

找错了、对不齐、还是执行不了，三个环节能分开查。

库可以一直攒

演示越多，越可能撞上差不多能用的那一条。

边界：单臂、开环、演示与部署时物体摆放不能差太多。

第二部分

02

这条线走了五年，真正变天只有一次。

前人走到哪一步

同样的 " 检索 " ，用法变了三次；每一次变化都重新定义了 " 这条路线要花多少代价 " 。

- 三代做法的分界线，不在 " 检索什么 " ，而在 " 检索到之后怎么用 " 。
- " 上下文 " 这个词，领域里至少指三件量纲不同的事——这是后面所有讨论的前提。
- 把八项代表工作摊在一张表上，能直接看出哪一格是空的。

> 先记住一个转折： 2021 年的做法是 " 检索一个动作 " ， 2025 年之后是 " 检索一段演示当上下文 "—— 同一个动作，含义完全不同。

三代人做的事，分界线在 " 检索完怎么用 "

同样是检索，第一代直接出动作，第二代要重训，第三代才真正做到不训练

2021
第一代

检索一个动作，直接输出

从库里找最像的几帧，把它们对应的动作加权平均，就是下一步要执行的指令。 代表工作是 VINN 。

代价：免训练，但只有单步、没有时序结构；在精细任务上终局成功率接近 0 。

2023–2025
第二代

检索到的数据，要一起训练

把检索出来的片段和目标任务的少量数据混在一起重新训练策略。 代表工作是 Behavior Retrieval 、 STRAP 、 IWR 。

代价：每个新任务仍要付一次训练；有工作部署时要重训约 35 分钟。

2025–2026
第三代

检索结果当上下文，喂给冻结的策略

检索到的演示不参与训练，直接作为提示交给一个不动的策略，动作由它现场生成。 代表工作是 MT3 、 ReCAP 。

代价：真正零训练；但只能吃定长片段，或一条完整演示。

所以 判断一项工作属于哪一代，不要看它检索什么，要看它检索完还训不训练。

" 上下文 " 这个词，被用来指三件不同的事

三句话在各自的量纲上都对，放在一起就没法比

单位：时间步

训练时，一条序列看多长

指预训练时把多长的历史压进一条训练样本。有工作把这个长度拉到 8000 步，30 赫兹下相当于四分半钟的连续操作。

8000 步

但下游微调只用到 1000 步——训练给的能力和部署用的长度是两回事。

单位：条

提示里，放几条演示

指推理时直接摆在模型面前、让它照着的演示数量。主流做法固定为一条，厂商对外展示时也只放一段第一视角视频。

通常 1 条

几乎没有人试过把这一条换成三条、五条，看会发生什么。

单位：条 / 任务

从多大的库里，取几条

指演示库的规模，以及每次实际取回多少条。有工作把取回条数从 100 扫到 4000，发现最优点随任务而变。

100 → 4000

这是唯一被系统扫过的一根轴，但库的大小和取几条是分开动的。

所以 " 这个方向还没人做全 " 这句话，必须先说清是哪一根轴。把三根轴混着说，谁都可以说 " 我做了上下文 "，也谁都无法被反驳。

谁调过哪一根轴

八项代表工作摊开看。前四列是自变量，最后一列是它要不要为每个新任务付训练成本。

> 读法提示：RoboTTT 那一格的 128 – 8000 是训练时的序列长度，不是检索条数，别和上面几行混着读。

● 系统扫过这根轴

— 固定不动或没有这根轴

代表工作	库多大	取几条	每条多长	免训练
VINN · 2021	● 930/750/71	• 约 10	—	是
Behavior Retrieval · 2023	—	—	—	否
STRAP · 2024	● 76k 条轨迹	● 100–4000	—	否
ReMoBot · 2024	● 1–20	● 5 / 10	● 3 / 5	是
MT3 · 2025	● 1–50	— 恒 1 条	— 单条完整	是
SAIL · 2026	—	● 1 / 3	—	是
ReCAP · 2026	● 11–35	• 取前 5	— 固定 16 帧	是
RoboTTT · 2026	—	—	● 128–8000	否

只有一项同时碰过前三列，而且只是四个格子的粗扫（取 5 条或 10 条 × 每条 3 段或 5 段）。没有任何一项在"总预算固定"的前提下画过等值线——这就是空位。

第三部分

03

两个发现和我们原本的直觉相反，第三个让现有的归因数字不可信。

三个没料到的发现

前两个是别人已经测出来、但没被当成问题讲的数字，第三个是我们自己算的。

- 演示给得越多，成功率不一定越高——三处独立证据都出现了下降。
- 领域默认的评测配置分辨不出五个百分点——大量 " 提升五个点 " 其实测不出来。
- 分割错、位姿错、检索错会互相伪装——论文里 " 失败原因占比 " 不能直接信。

共同点：这三件事都不是 " 做不出来 " ，而是 " 没人把它当成一件要测的事 " 。

三个发现指向同一件事：这条线上的数字，现在还不能被当成结论用。

发现一：演示给得越多，反而更差

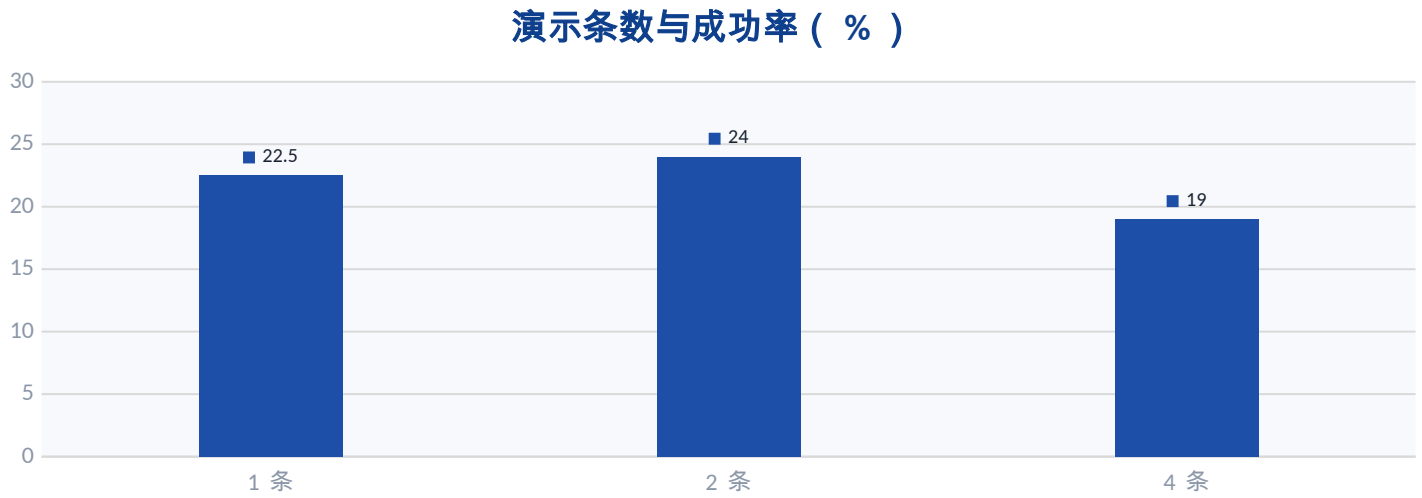
三处互相独立的工作里都出现了下降，说明这不是噪声，是有最优区间

第一处证据

演示 1 条→ 2 条→ 4 条

成功率 22.5 → 24.0 → **19.0** 。 其中 " 提起瓶子 " 这一项从 31 涨到 36 ，再掉到 **2** 。

数据来自一项触觉演示工作，附录里的消融实验。



第二处证据：给 1 帧历史反而更差

同一任务上，给一帧历史 **39.5%** ， 完全不给历史 **57%** 。

第三处证据：最优点随任务移动

取回条数从 100 扫到 4000 ，最优值在不同任务上不一样，不是 " 越大越好 " 。

这意味着：报 " 用了三条演示 " 这种单点结果没有意义，必须画出整条曲线， 并且说清最优点在哪里、从哪开始变坏。

发现二 · 我们自己算的

9.4

个百分点

这是通用仿真基准的默认配置能分辨的**最小差异**——比我们要测的效应还大。

默认配置是什么

20 个任务 × 每个任务跑 20 次 × 1 个随机种子。这个分辨率下，5 个百分点的差异只有 **30%** 的把握能测出来。

换成我们的配置

20 个任务 × 50 次 × 3 个种子 → 分辨率 **3.4 个百分点**，把握 **98%**。

成本

18 个配置约 10.8 万次试验，一台 4090 一夜到一天。

稳健性

九组不同假设下都落在 3.4 到 3.8 之间，别人可以复算。

所以：领域里大量 " 提升五个点 " 的结论，在这个默认配置下本来就测不出来。 我们自己的实验，必须先把分辨率讲清楚，再谈效果。

发现三：三种错误，会互相伪装

只看最后成没成，分不清是哪一环断的

一个分割错误，会被记成位姿错误

掩码混进了桌面，深度中位数就偏了；位姿模块拿到偏掉的初值，还会 " 自信地 " 收敛到一个错误答案——它看不到掩码是错的。

> 所以论文里 " 检索失败占 22.3%" 这类数字，如果是端到端打一个标签归出来的，就不能直接信。

唯一可靠的办法：拿真值掩码、真值位姿重跑一小批（比如 10% 的回合），看成功率怎么变。

真实原因	被当成	为什么分不开
分割错	位姿错	掩码混入桌面 → 深度初值偏移 → 位姿自信地收敛到错处
对称物体翻转	分割错	误差很大，但掩码其实是对的
检索到相似物	位姿错	对错误的物体估得很准，位姿分数反而更高
跟踪漂移	检索错	首帧其实是对的，错在后面的传播

要记的东西：每一帧的掩码面积与点数、位姿前两名假设的分数差、帧间旋转跳变、抓取位姿与成败。 这些量不记，事后只能靠猜。

第四部分

04

一　预算等值线

二　换算率协议

三　阶段级归因

三件事，标准是同一条：别人能不能照着复算。

接下来要做什么

不是再做一个更强的策略，而是把这条路上的三个数字变成别人能复算的东西。

- 固定总预算，画 " 取几条 × 每条多长 " 的等值线。
- 一条上下文演示相当于多少条训练示范——给出可复算的换算规则。
- 把 " 哪一步断了 " 变成带置信区间的估计，而不是事后人工归类。

三条底线：等算力、等数据、检索度量固定——少一条，曲线就不可解释。

> 这三件事都是零训练或低成本实验，不需要新硬件。

我们要补的三件测量

每一件都能落成一张图或一张表，别人拿着我们的数据和代码可以重算。

前两件管 " 给多少上下文 " ，第三件管 " 断在哪一环 " 。前者决定画什么图，后者决定这张图能不能解释。

共同前提：仿真里做，不需要新硬件。

一台显卡一夜到一天一轮。

一　预算等值线：在总预算固定下换组合

同样是 32 段上下文，可以是 " 取 4 条 × 每条 8 段 " ，也可以是 " 取 1 条 × 每条 32 段 " 。不固定总预算，" 多给几条 " 和 " 每条更长 " 就混在一起，分不清是谁的功劳。

二　换算率：一条演示顶多少条训练示范

必须同时报三样：插值规则、相邻测点之间的距离、交叉点的置信区间。 现在唯一公开的数字来自厂商博客，是插值估出来的——换一种插值方式，结果差 14% 。

三　阶段级归因：把 " 哪一步断了 " 变成区间估计

在评测的统计模型里加一个 " 阶段 " 项，输出 " 某方法在某阶段显著更差 " 以及它的置信区间； 再用真值掩码、真值位姿做反事实校验，防止三种错误互相伪装。

三条底线　等算力、等数据、检索度量固定——少一条，画出来的曲线就不可解释。

一句话总结

我们不是要做更强的策略，是要让这条路的强弱可被测量。

可归因，才谈得上 scaling ；可复算，才谈得上领先。

需要配合之一

一段连续的显卡机时，跑完等值线扫描。

需要配合之二

Sawyer 的机时窗口，做真机单演示适配。

需要配合之三

对 " 阶段判据 " 的评审意见——这一条不能自己说了算。